Fjalorët dhe Set në Python

Alion Emini

Kurs i Programimit në Python
kurse.informatika
November 5, 2025

Contents

1 Fjalorët dhe Set **5**

1.1 Fjalorët (Dictionaries) . . . 5

1.1.1 Krijimi i një fjalori . . . 5

1.1.2 Marrja e një vlere nga fjalori . . . 6

1.1.3 Përdorimi i operatorëve `in` dhe `not in` për të testuar një çelës në fjalor . . . 7

1.1.4 Shtimi i elementëve në një fjalor ekzistues . . . 8

1.1.5 Fshirja e elementëve . . . 9

1.1.6 Marrja e numrit të elementëve në një fjalor . . . 9

1.1.7 Përzierja e tipeve të të dhënave në një fjalor . . . 10

1.1.8 Krijimi i një fjalori bosh . . . 11

1.1.9 Përdorimi i ciklit `for` për të iteruar mbi një fjalor . . . 12

1.1.10 Disa metoda të fjalorit . . . 12

1.1.11 Shprehje me fjalor (*Dictionary Comprehensions*) . . . 17

1.1.12 Përdorimi i klauzolave `if` në *dictionary comprehensions* . . . 18

1.2 Set . . . 19

1.2.1 Krijimi i një *set*-i . . . 19

1.2.2 Marrja e numrit të elementëve në një *set* . . . 20

1.2.3 Shtimi dhe heqja e elementëve . . . 20

1.2.4 Përdorimi i ciklit `for` për të iteruar mbi një *set* . . . 22

1.2.5 Përdorimi i operatorëve `in` dhe `not in` për testim në *set* . . . 22

1.2.6 Gjetja e bashkimit të *set*-eve (*Union*) . . . 23

1.2.7 Gjetja e prerjes (*Intersection*) . . . 24

1.2.8 Gjetja e diferencës (*Difference*) . . . 25

1.2.9 Gjetja e diferencës simetrike (*Symmetric Difference*) . . . 25

1.2.10 Nënsete (*Subsets*) dhe mbisete (*Supersets*) . . . 26

1.2.11 Shprehje me *set* (*Set Comprehensions*) . . . 27

1.3 Fjalorë të folezuar (*Nested Dictionaries*) . . . 28

1.3.1 Krijimi dhe përditësimi . . . 29

1.3.2 Aksesim i sigurt me `get` . . . 29

1.3.3 Iterimi në nivele . . . 30

1.3.4 Fshirja në nivele . . . 30

1.3.5 *Nested* dictionary comprehensions . . . 30

1.4 Serializimi i objekteve (*Pickling*) . . . 30

1.4.1 Pse serializojmë? . . . 30

1.4.2 Hapat bazë për *pickling* . . . 31

1.4.3 Hapat bazë për *unpickling* . . . 31

1.5 Formati JSON dhe përdorimi në Python . . . 33

1.5.1 Hartimi i tipeve: JSON ↔ Python . . . 33

1.5.2 Serializim/deserializim bazë . . . 33

1.5.3 Punë me data/kohë dhe tipe jo-standarde 34
1.5.4 Saktësia numerike me `decimal.Decimal` 35
1.5.5 JSON me fjalorë të folezuar 35
1.5.6 Lexim i shpejtë i rrjedhave "një-JSON-për-rresht" (NDJSON) 35
1.5.7 Trajtimi i gabimeve 35

Chapter 1

Fjalorët dhe Set

1.1 Fjalorët (Dictionaries)

Përkufizim

Fjalor (*dictionary*) quhet një objekt që ruan një koleksion të dhënash. Çdo element në një fjalor ka dy pjesë: një **çelës** (*key*) dhe një **vlerë** (*value*). Përdor një çelës për të gjetur një vlerë specifike.

Kur dëgjon fjalën 'fjalor', me shumë gjasa të shkon mendja te një libër i madh si fjalori Merriam–Webster, që përmban fjalë dhe përkufizimet e tyre. Nëse do kuptimin e një fjale të caktuar, e gjen atë në fjalor për të parë përkufizimin.
Në Python, një **fjalor** është një objekt që ruan një koleksion të dhënash. Çdo element i ruajtur në një fjalor ka dy pjesë: një *çelës* dhe një *vlerë*. Në fakt, elementët e fjalorit referohen shpesh si **çifte çelës–vlerë** (*key–value pairs*). Kur dëshiron të marrësh një vlerë specifike nga një fjalor, përdor çelësin që është i lidhur me atë vlerë. Kjo i ngjan procesit të kërkimit të një fjale në fjalorin Merriam–Webster, ku fjalët janë çelësa dhe përkufizimet janë vlera.
Për shembull, supozo që çdo punonjës në një kompani ka një numër ID, dhe duam të shkruajmë një program që na lejon të kërkojmë emrin e një punonjësi duke futur ID-në e tij. Mund të krijojmë një fjalor ku çdo element përmban numrin e ID-së së punonjësit si *çelës*, dhe emrin e atij punonjësi si *vlerë*. Nëse dimë numrin e ID-së, mund të marrim emrin e punonjësit.
Një shembull tjetër do të ishte një program që na lejon të fusim emrin e një personi dhe na jep numrin e tij të telefonit. Programi mund të përdorë një fjalor ku çdo element përmban emrin e personit si *çelës*, dhe numrin e telefonit të atij personi si *vlerë*. Nëse dimë emrin e një personi, mund të marrim numrin e tij të telefonit.

Shënim

Çiftet *çelës–vlerë* shpesh quhen edhe **mappings**, sepse çdo çelës *mapohet* te një vlerë.

1.1.1 Krijimi i një fjalori

Mund të krijosh një fjalor duke i futur elementët brenda një palë kllapash kaçurrelë `{ }`. Një element përbëhet nga një *çelës*, i pasuar nga dy pika `:`, i pasuar nga *vlera*. Elementët ndahen me presje. Deklarata në vijim tregon një shembull:

```
phonebook = {'Chris':'555-1111', 'Katie':'555-2222', 'Joanne':'555-3333'}
```

Kjo deklaratë krijon një fjalor dhe ia cakton atë variablës `phonebook`. Fjalori përmban këto tre elementë:

- Elementi i parë është `'Chris':'555-1111'`. Këtu çelësi është `'Chris'` dhe vlera është `'555-1111'`.

- Elementi i dytë është `'Katie':'555-2222'`. Këtu çelësi është `'Katie'` dhe vlera është `'555-2222'`.
- Elementi i tretë është `'Joanne':'555-3333'`. Këtu çelësi është `'Joanne'` dhe vlera është `'555-3333'`.

Në këtë shembull, çelësat dhe vlerat janë *string*-je. Vlerat në një fjalor mund të jenë objekte të çfarëdoshme tipi, por **çelësat duhet të jenë objekte të pandryshueshme** (*immutable*). Për shembull, çelësat mund të jenë *string*-je, *integers*, numra me presje lëvizëse, ose *tuple*. **Çelësat nuk mund të jenë lista** ose ndonjë tjetër tip *i ndryshueshëm*.

Sqarim i domosdoshëm

Në Python, *çelësat duhet të jenë të pandryshueshëm (immutable)* dhe *nuk* mund të jenë tipe *të ndryshueshme (mutable)* si listat.

1.1.2 Marrja e një vlere nga fjalori

Elementët në një fjalor **nuk** ruhen sipas një rendi të caktuar të indeksuar si në sekuenca. Për shembull, shiko sesionin e mëposhtëm interaktiv ku krijohet një fjalor dhe shfaqen elementët e tij:

```
phonebook = {'Chris':'555-1111', 'Katie':'555-2222', 'Joanne':'555-3333'}
print(phonebook)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
{'Joanne': '555-3333', 'Chris': '555-1111', 'Katie': '555-2222'}
```

Vër re që rendi në të cilin shfaqen elementët është ndryshe nga rendi në të cilin u krijuan. Kjo tregon se fjalorët *nuk janë sekuenca* si listat, tuplet dhe stringjet. Si rezultat, **nuk** mund të përdorësh një indeks numerik për të marrë një vlerë sipas pozicionit nga një fjalor. Në vend të kësaj, përdor një *çelës* për të marrë një vlerë.

Shënim modern i dobishëm

Që nga Python 3.7, fjalorët ruajnë *rendin e futjes* gjatë iterimit. Megjithatë, fjalori nuk është strukturë e indeksuar si listat dhe *nuk* duhet përdorur për aksesim me indeks numerik.

Për të marrë një vlerë nga një fjalor, thjesht shkruan një shprehje me formatin e përgjithshëm:

```
dictionary_name[key]
```

Në këtë format, `dictionary_name` është variabla që referon fjalorin dhe `key` është një çelës. Nëse çelësi ekziston në fjalor, shprehja kthen vlerën që është e lidhur me të. Nëse çelësi nuk ekziston, ngrihet përjashtimi `KeyError`. Sesioni i mëposhtëm e demonstron këtë:

```
phonebook = {'Chris':'555-1111', 'Katie':'555-2222', 'Joanne':'555-3333'}
print(phonebook['Chris'])
print(phonebook['Joanne'])
print(phonebook['Katie'])
print(phonebook['Kathryn'])
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
555-1111
555-3333
555-2222
Traceback (most recent call last):
  File "script.py", line 4, in <module>
    print(phonebook['Kathryn'])
KeyError: 'Kathryn'
```

Le ta shohim më nga afër sesionin:

- **Rreshti 1** krijon një fjalor që përmban emra (si çelësa) dhe numra telefoni (si vlera).
- **Në rreshtin 2**, shprehja `phonebook['Chris']` kthen vlerën nga fjalori `phonebook` që është e lidhur me çelësin `'Chris'`. Vlera shfaqet në rreshtin 3.
- **Në rreshtin 4**, shprehja `phonebook['Joanne']` kthen vlerën e lidhur me çelësin `'Joanne'`. Vlera shfaqet në rreshtin 5.
- **Në rreshtin 6**, shprehja `phonebook['Katie']` kthen vlerën e lidhur me çelësin `'Katie'`. Vlera shfaqet në rreshtin 7.
- **Në rreshtin 8**, shkruhet `phonebook['Kathryn']`. Nuk ka një çelës `'Kathryn'` në fjalorin `phonebook`, ndaj ngrihet përjashtimi `KeyError`.

Shënim

Mbaj mend që krahasimet e *string*-jeve janë **sensibile ndaj madhësisë së shkronjave** (*case sensitive*). Shprehja `phonebook['katie']` nuk do të gjejë çelësin `'Katie'` në fjalor.

1.1.3 Përdorimi i operatorëve `in` dhe `not in` për të testuar një çelës në fjalor

Siç u demonstrua më sipër, ngrihet një `KeyError` nëse tenton të marrësh një vlerë nga një fjalor duke përdorur një çelës që nuk ekziston. Për të parandaluar një përjashtim të tillë, mund të përdorësh operatorin `in` për të përcaktuar nëse një çelës ekziston përpara se ta përdorësh atë për të marrë një vlerë. Sesioni i mëposhtëm e demonstron:

```
phonebook = {'Chris':'555-1111', 'Katie':'555-2222', 'Joanne':'555-3333'}
if 'Chris' in phonebook:
    print(phonebook['Chris'])
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
555-1111
```

Deklarata `if` në rreshtin 2 përcakton nëse çelësi `'Chris'` gjendet në fjalorin `phonebook`. Nëse po, deklarata në rreshtin 3 shfaq vlerën që është e lidhur me atë çelës. Mund të përdorësh gjithashtu operatorin `not in` për të përcaktuar nëse një çelës *nuk* ekziston, siç tregohet në sesionin vijues:

```
phonebook = {'Chris':'555-1111', 'Katie':'555-2222'}
if 'Joanne' not in phonebook:
    print('Joanne is not found')
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
Joanne is not found
```

Shënim

Kujto se krahasimet e *string*-jeve me operatorët `in` dhe `not in` janë **sensibile ndaj madhësisë së shkronjave** (*case sensitive*).

1.1.4 Shtimi i elementëve në një fjalor ekzistues

Fjalorët janë objekte *mutable*. Mund të shtosh çifte të reja *çelës–vlerë* me një deklaratë caktimi me formatin e përgjithshëm:

```
dictionary_name[key] = value
```

Në këtë format, `dictionary_name` është variabla që referon fjalorin, dhe `key` është një çelës. Nëse `key` ekziston tashmë në fjalor, vlera e tij do të zëvendësohet me `value`. Nëse çelësi nuk ekziston, do të *shtohet* në fjalor bashkë me vlerën e tij. Shembulli i mëposhtëm e demonstron:

```
phonebook = {'Chris':'555-1111', 'Katie':'555-2222', 'Joanne':'555-3333'}
phonebook['Joe'] = '555-0123' # shto çelës të ri
phonebook['Chris'] = '555-4444' # ndrysho vlerën e një çelësi ekzistues
print(phonebook)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
{'Chris': '555-4444', 'Joanne': '555-3333', 'Joe': '555-0123', 'Katie': '555-2222'}
```

Shpjegim i shkurtër:

- **Rreshti 1** krijon fjalorin me emra (çelësa) dhe numra telefoni (vlera).
- **Rreshti 2** shton çiftin e ri `'Joe':'555-0123'` (çelësi s'ekzistonte).
- **Rreshti 3** zëvendëson vlerën e çelësit ekzistues `'Chris'` me `'555-4444'`.
- **Rreshti 4** shfaq përmbajtjen e fjalorit.

Shënim

Nuk mund të kesh **çelësa të dublikuar** në një fjalor. Nëse i cakton një vlerë një çelësi ekzistues, vlera e re *zëvendëson* vlerën e vjetër.

1.1.5 Fshirja e elementëve

Mund të fshish një çift ekzistues *çelës–vlerë* me fjalën kyçe `del`. Formati i përgjithshëm:

```
del dictionary_name[key]
```

Pas ekzekutimit, `key` dhe vlera e tij hiqen nga fjalori. Nëse çelësi nuk ekziston, ngrihet `KeyError`. Shembull:

```
phonebook = {'Chris':'555-1111', 'Katie':'555-2222', 'Joanne':'555-3333'}
print(phonebook)
del phonebook['Chris']
print(phonebook)
del phonebook['Chris'] # provë për ta fshirë sërish
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
{'Chris': '555-1111', 'Joanne': '555-3333', 'Katie': '555-2222'}
{'Joanne': '555-3333', 'Katie': '555-2222'}
Traceback (most recent call last):
  File "script.py", line 7, in <module>
    del phonebook['Chris']
KeyError: 'Chris'
```

Për të shmangur `KeyError`, kontrollo fillimisht ekzistencën e çelësit me `in`:

```
phonebook = {'Chris':'555-1111', 'Katie':'555-2222', 'Joanne':'555-3333'}
if 'Chris' in phonebook:
    del phonebook['Chris']

print(phonebook)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
{'Joanne': '555-3333', 'Katie': '555-2222'}
```

1.1.6 Marrja e numrit të elementëve në një fjalor

Përdor funksionin e brendshëm `len` për të marrë numrin e elementëve:

```
phonebook = {'Chris':'555-1111', 'Katie':'555-2222'}
num_items = len(phonebook)
print(num_items)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
2
```

Shpjegim:

- **Rreshti 1** krijon një fjalor me dy elementë.
- **Rreshti 2** thërret `len(phonebook)` dhe e ruan rezultatin në `num_items`.
- **Rreshti 3** shfaq vlerën `2`.

1.1.7 Përzierja e tipeve të të dhënave në një fjalor

Siç u përmend, **çelësat duhet të jenë *immutable***, por vlerat mund të jenë të çdo tipi. P.sh., vlerat mund të jenë lista:

```
test_scores = {
    'Kayla': [88, 92, 100],
    'Luis': [95, 74, 81],
    'Sophie': [72, 88, 91],
    'Ethan': [70, 75, 78]
}

print(test_scores)
print(test_scores['Sophie'])

kayla_scores = test_scores['Kayla']
print(kayla_scores)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
{'Kayla': [88, 92, 100], 'Sophie': [72, 88, 91], 'Ethan': [70, 75, 78], 'Luis': [95, 74, 81]}
[72, 88, 91]
[88, 92, 100]
```

Fjalori përmban katër elementë:

- `'Kayla' : [88, 92, 100]` — çelës `'Kayla'`, vlerë lista `[88, 92, 100]`.
- `'Luis' : [95, 74, 81]` — çelës `'Luis'`, vlerë lista `[95, 74, 81]`.
- `'Sophie': [72, 88, 91]` — çelës `'Sophie'`, vlerë lista `[72, 88, 91]`.
- `'Ethan' : [70, 75, 78]` — çelës `'Ethan'`, vlerë lista `[70, 75, 78]`.

Vlerat brenda *të njëjtit* fjalor mund të jenë edhe të tipeve të ndryshme; po ashtu edhe çelësat (sa kohë që janë *immutable*).

Shembull — Fjalor me tipe të përziera të të dhënave

```
mixed_up = {'abc': 1, 999: 'yada yada', (3, 6, 9): [3, 6, 9]}
print(mixed_up)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
{(3, 6, 9): [3, 6, 9], 'abc': 1, 999: 'yada yada'}
```

Ky fjalor përmban:

- `'abc': 1` — çelës *string*, vlerë *integer*.
- `999: 'yada yada'` — çelës *integer*, vlerë *string*.
- `(3, 6, 9): [3, 6, 9]` — çelës *tuple* (immutable), vlerë *listë*.

Shembull — Të dhëna për një punonjës

Shembull më praktik: të dhëna të ndryshme për një punonjës:

```
employee = {'name': 'Kevin Smith', 'id': 12345, 'payrate': 25.75}
print(employee)
```

```
{'payrate': 25.75, 'name': 'Kevin Smith', 'id': 12345}
```

Elementet janë:

- `'name': 'Kevin Smith'` — çelës dhe vlerë *string*.
- `'id': 12345` — çelës *string*, vlerë *integer*.
- `'payrate': 25.75` — çelës *string*, vlerë *float*.

1.1.8 Krijimi i një fjalori bosh

Ndonjëherë ke nevojë të nisësh me një fjalor bosh dhe ta mbushësh gjatë ekzekutimit:

```
phonebook = {}
phonebook['Chris'] = '555-1111'
phonebook['Katie'] = '555-2222'
phonebook['Joanne'] = '555-3333'
print(phonebook)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
{'Chris': '555-1111', 'Joanne': '555-3333', 'Katie': '555-2222'}
```

Mund të përdorësh edhe funksionin e brendshëm `dict()` për të krijuar një fjalor bosh:

```
phonebook = dict()
```

1.1.9 Përdorimi i ciklit for për të iteruar mbi një fjalor

Mund të përdorësh ciklin `for` në formatin e përgjithshëm më poshtë për të iteruar mbi të *gjithë çelësat* e një fjalori:

```
for var in dictionary:
    statement
    statement
    # etj.
```

Në këtë format, `var` është emri i një variable dhe `dictionary` është emri i fjalorit. Cikli iteron një herë për çdo element të fjalorit. Çdo iterim i cakton `var`-it një *çelës*. Sesioni i mëposhtëm e demonstron:

```
phonebook = {
    'Chris': '555-1111',
    'Katie': '555-2222',
    'Joanne': '555-3333'
}

for key in phonebook:
    print(key)

for key in phonebook:
    print(key, phonebook[key])
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
Chris
Joanne
Katie
Chris 555-1111
Joanne 555-3333
Katie 555-2222
```

Përmbledhje e deklaratave:

- **Rreshtat 1–5** krijojnë një fjalor me tre elementë dhe ia caktojnë atë variablës `phonebook`.
- **Rreshtat 7–8** përmbajnë një `for` që iteron një herë për çdo element të `phonebook`. Çdo herë `key` merr një çelës dhe `print` shfaq vlerën e `key`. Rreshtat e daljes shfaqen më sipër.
- **Rreshtat 10–11** përmbajnë një `for` tjetër që, për çdo element, shfaq çelësin dhe më pas vlerën e lidhur me të, me `phonebook[key]`.

1.1.10 Disa metoda të fjalorit

Objektet *dictionary* kanë disa metoda. Në këtë seksion shohim disa prej më të dobishmeve, të përmbledhura në Tabelën 1.1.

Shënim modern

Në Python 3, `items()`, `keys()` dhe `values()` kthejnë *dictionary views* (jo lista të zakonshme). `items()` është një view me `(key, value)` *tuples*, `keys()` një view me çelësa, dhe `values()` një view me vlera.

Table 1.1: Disa nga metodat e fjalorit

Metoda	Përshkrimi
`clear`	Zbraz përmbajtjen e fjalorit (e lë të zbrazët).
`get`	Merr vlerën e lidhur me një çelës të caktuar. Nëse çelësi nuk gjendet, metoda *nuk* ngre përjashtim; në vend të kësaj kthen një vlerë të parazgjedhur.
`items`	Kthen të gjithë çelësat e një fjalori *dhe* vlerat e tyre të lidhura si një renditje *tuplesh*.
`keys`	Kthen të gjithë çelësat e një fjalori si një renditje *tuplesh*.
`pop`	Kthen vlerën e lidhur me një çelës të caktuar *dhe* e heq atë çift nga fjalori. Nëse çelësi nuk gjendet, metoda kthen një vlerë të parazgjedhur.
`popitem`	Kthen, si *tuple*, çiftin çelës–vlerë që u shtua së fundi në fjalor. Metoda gjithashtu e heq atë çift nga fjalori.
`values`	Kthen të gjitha vlerat në fjalor si një renditje *tuplesh*.

Metoda `clear`

`clear` fshin të gjithë elementët e një fjalori, duke e lënë bosh. Formati i përgjithshëm:

```
dictionary.clear()
```

Shembull:

```
phonebook = {'Chris':'555-1111', 'Katie':'555-2222'}
print(phonebook)
phonebook.clear()
print(phonebook)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
{'Chris': '555-1111', 'Katie': '555-2222'}
{}
```

Vër re se pas thirrjes së `clear()`, fjalori `phonebook` nuk përmban elementë.

Metoda `get`

`get` mund të përdoret si alternativë e operatorit `[]` për marrjen e një vlere nga fjalori. `get` *nuk* ngre përjashtim nëse çelësi nuk gjendet. Formati i përgjithshëm:

```
dictionary.get(key, default)
```

Ku `dictionary` është emri i fjalorit, `key` është çelësi që kërkohet dhe `default` është vlera që kthehet nëse çelësi nuk gjendet.

```
phonebook = {'Chris':'555-1111', 'Katie':'555-2222'}

value = phonebook.get('Katie', 'Entry not found')
print(value)

```

```
6 value = phonebook.get('Andy', 'Entry not found')
7 print(value)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
1 555-2222
2 Entry not found
```

Përmbledhje:

- Rreshti 3 kërkon çelësin `'Katie'` — gjehet dhe vlera i caktohet `value`.
- Rreshti 6 kërkon çelësin `'Andy'` — nuk gjendet, ndaj `'Entry not found'` i caktohet `value`.

Metoda `items`

`items` kthen të gjithë çelësat dhe vlerat e fjalorit si një *dictionary view*, ku çdo element është një *tuple* `(key, value)`.

```
1 phonebook = {
2     'Chris': '555-1111',
3     'Katie': '555-2222',
4     'Joanne': '555-3333'
5 }
6
7 for key, value in phonebook.items():
8     print(key, value)
```

```
1 Chris 555-1111
2 Joanne 555-3333
3 Katie 555-2222
```

Përmbledhje:

- Cikli `for` thërret `phonebook.items()`, merr një renditje tuple-sh dhe iteron një herë për çdo tuple.
- Çdo herë, vlerat e tuple-it u caktohen variablave `key` dhe `value`, të cilat shtypen me `print`.

Metoda keys

`keys` kthen të gjithë çelësat e fjalorit si një *dictionary view* (renditje çelësash).

```
1 phonebook = {
2     'Chris': '555-1111',
3     'Katie': '555-2222',
4     'Joanne': '555-3333'
5 }
6
7 for key in phonebook.keys():
8     print(key)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
Chris
Joanne
Katie
```

Metoda pop

`pop` kthen vlerën e lidhur me një çelës të caktuar *dhe* e heq çiftin nga fjalori. Nëse çelësi nuk gjendet, kthen një vlerë të parazgjedhur.
Formati i përgjithshëm:

```
dictionary.pop(key, default)
```

```
phonebook = {
    'Chris': '555-1111',
    'Katie': '555-2222',
    'Joanne': '555-3333'
}

phone_num = phonebook.pop('Chris', 'Entry not found')
print(phone_num)
print(phonebook)

phone_num = phonebook.pop('Andy', 'Element not found')
print(phone_num)
print(phonebook)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
555-1111
{'Joanne': '555-3333', 'Katie': '555-2222'}
Element not found
{'Joanne': '555-3333', 'Katie': '555-2222'}
```

Përmbledhje:

- Thirrja e parë e `pop('Chris', ...)` kthen vlerën e `'Chris'` dhe e *heq* çiftin përkatës nga fjalori.
- Thirrja e dytë kërkon `'Andy'` — nuk gjendet, ndaj kthehet mesazhi i paracaktuar.

Metoda popitem

`popitem` kryen dy veprime: (1) heq nga fjalori çiftin *çelës–vlerë* që u shtua *së fundi*, dhe (2) e kthen atë çift si një *tuple*. Formati i përgjithshëm:

```
dictionary.popitem()
```

Mund të përdorësh një caktim të shumëfishtë për të marrë çelësin dhe vlerën në variabla të veçantë:

```
k, v = dictionary.popitem()
```

Pas ekzekutimit, `k` merr çelësin dhe `v` merr vlerën e atij çelësi. Çifti hiqet nga fjalori.

```
phonebook = {
    'Chris': '555-1111',
    'Katie': '555-2222',
    'Joanne': '555-3333'
}

print(phonebook)
key, value = phonebook.popitem()
print(key, value)
print(phonebook)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
{'Chris': '555-1111', 'Joanne': '555-3333', 'Katie': '555-2222'}
Chris 555-1111
{'Joanne': '555-3333', 'Katie': '555-2222'}
```

Kujdes

Në Python 3.7+, `popitem()` është *LIFO* (heq çiftin e *fundit* të futur). Thirrja e `popitem()` mbi një fjalor *bosh* ngre `KeyError`.

Metoda values

`values` kthen të gjitha vlerat (pa çelësit) si një *dictionary view*. Çdo element i view-t është një *vlerë* nga fjalori.

```
phonebook = {
    'Chris': '555-1111',
    'Katie': '555-2222',
    'Joanne': '555-3333'
}

for val in phonebook.values():
    print(val)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
555-1111
555-3333
555-2222
```

1.1.11 Shprehje me fjalor (*Dictionary Comprehensions*)

Dictionary comprehension është një shprehje që lexon një renditje elementësh hyrës dhe, duke i përpunuar, prodhon një *fjalor*. Ajo është e ngjashme me *list comprehensions* (shih Kap. 7), por rezultat prodhon çifte *çelës–vlerë*.

Shembull — Dictionary comprehension: shembull

Shembull duke marrë listën e mëposhtme të numrave

```
numbers = [1, 2, 3, 4]
```

duam të krijojmë një fjalor ku *çelësat* janë vetë elementët e `numbers`, dhe *vlerat* janë katrorët e tyre. Pra synojmë:

```
{1: 1, 2: 4, 3: 9, 4: 16}
```

Pa *comprehension*, ka shumë gjasa të shkruanim:

```
squares = {}
for item in numbers:
    squares[item] = item**2
```

Me një **dictionary comprehension** e arrijmë *në një rresht*:

```
squares = {item: item**2 for item in numbers}
```

Pjesët e një *dictionary comprehension*

Forma e përgjithshme është `{result_expr for target in iterable}`, ku për fjalorë `result_expr` është `key_expr : value_expr`.

Në shembullin `squares = {item: item**2 for item in numbers}`:

- **Shprehja e rezultatit** (`result expression`): `item: item**2`
- **Shprehja e iterimit** (`iteration expression`): `for item in numbers`

Çdo iterim ndërton një element të fjalorit me `key = item` dhe `value = item**2`.

```
numbers = [1, 2, 3, 4]
squares = {item: item**2 for item in numbers}

print(numbers)
print(squares)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
[1, 2, 3, 4]
{1: 1, 2: 4, 3: 9, 4: 16}
```

Përdorimi i një fjalori ekzistues si hyrje.

Mund të përdorim edhe një fjalor ekzistues si burim hyrjeje. Supozojmë fjalorin:

```
phonebook = {'Chris': '555-1111', 'Katie': '555-2222', 'Joanne': '555-3333'}
```

Për të krijuar një kopje me *dictionary comprehension*:

```
phonebook_copy = {k: v for (k, v) in phonebook.items()}
```

Këtu:

- **Shprehja e rezultatit**: `k: v`
- **Shprehja e iterimit**: `for k, v in phonebook.items()`

Vër re se `phonebook.items()` kthen një renditje *tuplesh* `(key, value)`, p.sh.:

```
[('Chris', '555-1111'), ('Katie', '555-2222'), ('Joanne', '555-3333')]
```

Shembull — Kopjimi i fjalorit me dictionary comprehension

```
phonebook = {'Chris': '555-1111', 'Katie': '555-2222', 'Joanne': '555-3333'}
phonebook_copy = {k: v for (k, v) in phonebook.items()}

print(phonebook)
print(phonebook_copy)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
{'Chris': '555-1111', 'Katie': '555-2222', 'Joanne': '555-3333'}
{'Chris': '555-1111', 'Katie': '555-2222', 'Joanne': '555-3333'}
```

1.1.12 Përdorimi i klauzolave `if` në *dictionary comprehensions*

Shpesh duam të përzgjedhim vetëm disa elementë gjatë përpunimit. Supozo fjalorin:

```
populations = {
    'New York': 8398748,
    'Los Angeles': 3990456,
```

```
    'Chicago': 2705994,
    'Houston': 2325502,
    'Phoenix': 1660272,
    'Philadelphia': 1584138
}
```

Duam një fjalor të ri që kopjon vetëm qytetet me popullsi > 2 000 000. Me një `for` të zakonshëm:

```
largest = {}
for k, v in populations.items():
    if v > 2_000_000:
        largest[k] = v
```

E njëjta gjë me një **dictionary comprehension** duke shtuar një `if` filtrues në fund:

```
largest = {k: v for k, v in populations.items() if v > 2000000}
```

Pas ekzekutimit, `largest` përmban:

```
{'New York': 8398748, 'Los Angeles': 3990456, 'Chicago': 2705994, 'Houston': 2325502}
```

1.2 Set

Përkufizim

Një **set** përmban një koleksion *vlerash unike* dhe funksionon si një *set matematikor*.
Ja disa gjëra të rëndësishme për *set*-et:

- Të gjithë elementët në një *set* duhet të jenë unikë. As dy elementë nuk mund të kenë të njëjtën vlerë.
- *Set*-et janë *të parenditura* — elementët nuk ruhen sipas ndonjë rendi të caktuar.
- Elementët e ruajtur në një *set* mund të jenë të tipeve të ndryshme të të dhënave.

1.2.1 Krijimi i një *set*-i

Për të krijuar një *set*, thërret funksionin e brendshëm `set`. Shembull për të krijuar një *set* bosh:

```
myset = set()
```

Pas kësaj deklarate, variabla `myset` referon një *set* bosh. Mund të kalosh edhe një *argument* te funksioni `set`. Argumenti duhet të jetë një objekt *iterable* (p.sh. listë, *tuple*, ose *string*). Elementët individualë të atij objekti bëhen elementë të *set*-it.

```
myset = set(['a', 'b', 'c'])
print(myset) # {'a', 'b', 'c'} (rendi mund të ndryshojë)
```

Nëse kalon një *string* si argument, çdo karakter individual në *string* bëhet anëtar i *set*-it:

```
myset = set('abc')
print(myset) # {'a', 'b', 'c'} (rendi mund të ndryshojë)
```

Set-et nuk mund të përmbajnë duplikime. Nëse i kalon `set`-it një argument që përmban elementë të dublikuar, vetëm *një* kopje e secilit element do të ruhet:

```
myset = set('aaabc')
print(myset) # {'a', 'b', 'c'} (rendi mund të ndryshojë)
```

Po sikur të duam të krijojmë një *set* ku *çdo element* është një *string* me më shumë se një karakter? P.sh. një *set* që përmban `'one'`, `'two'` dhe `'three'`.

```
# This is error: set accepts only one argument
myset = set('one', 'two', 'three')
```

```
# This does not make what we want
myset = set('one two three')
print(myset) # {'o', 'n', 'e', ' ', 't', 'w', 'h', 'r'} (karaktere individuale)
```

Për të marrë një *set* me `'one'`, `'two'`, `'three'`, duhet t'i kalojmë `set`-it një *listë* me këto *string*-je:

```
# This is correct
myset = set(['one', 'two', 'three'])
print(myset) # {'one', 'two', 'three'} (rendi mund të ndryshojë)
```

1.2.2 Marrja e numrit të elementëve në një *set*

Ashtu si te listat, *tuple*-t dhe fjalorët, mund të përdorësh `len` për të marrë numrin e elementëve të një *set*-i:

```
myset = set([1, 2, 3, 4, 5])
print(len(myset)) # 5
```

1.2.3 Shtimi dhe heqja e elementëve

Set-et janë objekte *mutable*, ndaj mund t'u shtosh ose t'u heqësh elementë.

Shtimi me `add`.

Përdor `add` për të shtuar një element:

```
myset = set()
myset.add(1)
myset.add(2)
myset.add(3)
print(myset) # {1, 2, 3}

myset.add(2) # 2 ekziston; s'ngre përjashtim, thjesht s'e shton
print(myset) # {1, 2, 3}
```

Shtimi në grup me `update`.

Përdor `update` për të shtuar disa elementë njëherësh nga një objekt *iterable* (listë, *tuple*, *string*, ose *set* tjetër):

```
myset = set([1, 2, 3])
myset.update([4, 5, 6])
print(myset) # {1, 2, 3, 4, 5, 6}
```

```
set1 = set([1, 2, 3])
set2 = set([8, 9, 10])
set1.update(set2)
print(set1) # {1, 2, 3, 8, 9, 10}
print(set2) # {8, 9, 10} (mbetet i pandryshuar)
```

```
myset = set([1, 2, 3])
myset.update('abc') # shton çdo karakter si element më vete
print(myset) # {1, 2, 3, 'a', 'b', 'c'} (rendi mund të ndryshojë)
```

Heqja me `remove` dhe `discard`.

Të dyja marrin si argument elementin për t'u hequr. Dallimi është te sjellja kur elementi *nuk* gjendet: `remove` ngre `KeyError`, ndërsa `discard` *jo*.

```
myset = set([1, 2, 3, 4, 5])
print(myset) # {1, 2, 3, 4, 5}

myset.remove(1)
print(myset) # {2, 3, 4, 5}

myset.discard(5)
print(myset) # {2, 3, 4}

myset.discard(99) # s'ngre përjashtim edhe nëse s'ekziston

myset.remove(99) # ngre KeyError
```

Gjurmë përjashtimi tipike:

```
Traceback (most recent call last):
  File "script.py", line N, in <module>
    myset.remove(99)
KeyError: 99
```

Zbrazja me `clear`.

`clear` heq të gjithë elementët e *set*-it:

```
myset = set([1, 2, 3, 4, 5])
print(myset) # {1, 2, 3, 4, 5}
myset.clear()
print(myset) # set()
```

Vër re: kur shfaqet një *set* bosh, interpretuese tregon `set()`.

1.2.4 Përdorimi i ciklit `for` për të iteruar mbi një *set*

Mund të përdorësh `for` në formatin e përgjithshëm:

```
for var in myset:
    statement
    statement
    # etj.
```

Cikli iteron një herë për çdo element të *set*-it, duke i caktuar `var`-it nga një element në çdo hap.

```
myset = set(['a', 'b', 'c'])
for val in myset:
    print(val)
```

Output i mundshëm (rendi s'garantohet):

```
a
c
b
```

1.2.5 Përdorimi i operatorëve `in` dhe `not in` për testim në *set*

Testim me `in`.

Përdor `in` për të përcaktuar nëse një vlerë ekziston në një *set*:

```
myset = set([1, 2, 3])
if 1 in myset:
```

```
    print('The value 1 is in the set.')
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
The value 1 is in the set.
```

Testim me `not in`.

Përdor `not in` për të përcaktuar nëse një vlerë *nuk* ekziston:

```
myset = set([1, 2, 3])
if 99 not in myset:
    print('The value 99 is not in the set.')
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
The value 99 is not in the set.
```

1.2.6 Gjetja e bashkimit të *set*-eve (*Union*)

Bashkimi i dy *set*-eve është një *set* që përmban të gjithë elementët e të dyjave. Në Python mund të thërrasësh metodën `union` për të marrë bashkimin e dy *set*-eve. Formati i përgjithshëm:

```
set1.union(set2)
```

Këtu `set1` dhe `set2` janë *set*-e. Metoda kthen një *set* që përmban elementët e të dy `set1` dhe `set2` (pa duplikime). Sesioni vijues e demonstron:

```
set1 = set([1, 2, 3, 4])
set2 = set([3, 4, 5, 6])
set3 = set1.union(set2)
print(set3)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
{1, 2, 3, 4, 5, 6}
```

Mund të përdorësh gjithashtu operatorin `|` për të gjetur bashkimin e dy *set*-eve. Formati i përgjithshëm:

```
set1 | set2
```

Shprehja kthen një *set* me elementët e të dy `set1` dhe `set2`. Demonstrim:

```
set1 = set([1, 2, 3, 4])
set2 = set([3, 4, 5, 6])
set3 = set1 | set2
print(set3)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
{1, 2, 3, 4, 5, 6}
```

1.2.7 Gjetja e prerjes (*Intersection*)

Prerja e dy *set*-eve është një *set* që përmban vetëm elementët e përbashkët të të dyjave. Përdor metodën `intersection` për të marrë prerjen. Formati i përgjithshëm:

```
set1.intersection(set2)
```

Metoda kthen një *set* që përmban elementët që gjenden si në `set1`, ashtu edhe në `set2`. Shembull:

```
set1 = set([1, 2, 3, 4])
set2 = set([3, 4, 5, 6])
set3 = set1.intersection(set2)
print(set3)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
{3, 4}
```

Mund të përdorësh edhe operatorin `&` për të gjetur prerjen. Formati i përgjithshëm:

```
set1 & set2
```

Shprehja kthen *set*-in e elementëve të përbashkët. Demonstrim:

```
set1 = set([1, 2, 3, 4])
set2 = set([3, 4, 5, 6])
set3 = set1 & set2
print(set3)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
{3, 4}
```

1.2.8 Gjetja e diferencës (*Difference*)

Diferenca `set1 - set2` janë elementët që shfaqen në `set1`, por *jo* në `set2`. Përdor metodën `difference` për ta marrë. Formati i përgjithshëm:

```
set1.difference(set2)
```

Metoda kthen *set*-in e elementëve që gjenden në `set1`, por jo në `set2`. Shembull:

```
set1 = set([1, 2, 3, 4])
set2 = set([3, 4, 5, 6])
set3 = set1.difference(set2)
print(set3)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
{1, 2}
```

Mund të përdorësh edhe operatorin `-` për të gjetur diferencën:

```
set1 - set2
```

Shembull:

```
set1 = set([1, 2, 3, 4])
set2 = set([3, 4, 5, 6])
set3 = set1 - set2
print(set3)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
{1, 2}
```

1.2.9 Gjetja e diferencës simetrike (*Symmetric Difference*)

Diferenca simetrike e dy *set*-eve është *set*-i i elementëve që *nuk* janë të përbashkët — pra elementët që janë në njërin *set* por jo në të dy. Përdor metodën `symmetric_difference`. Formati i përgjithshëm:

```
set1.symmetric_difference(set2)
```

Metoda kthen elementët që gjenden në `set1` *ose* `set2`, por jo në të dy. Shembull:

```
set1 = set([1, 2, 3, 4])
set2 = set([3, 4, 5, 6])
set3 = set1.symmetric_difference(set2)
print(set3)
```

```
{1, 2, 5, 6}
```

Mund të përdorësh edhe operatorin `^` për të gjetur diferencën simetrike:

```
set1 ^ set2
```

Shembull:

```
set1 = set([1, 2, 3, 4])
set2 = set([3, 4, 5, 6])
set3 = set1 ^ set2
print(set3)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
{1, 2, 5, 6}
```

1.2.10 Nënsete (*Subsets*) dhe mbisete (*Supersets*)

Supozo se ke dy *set*-e, dhe njëri prej tyre përmban të gjithë elementët e tjetrit:

```
set1 = set([1, 2, 3, 4])
set2 = set([2, 3])
```

Në këtë shembull, `set1` përmban të gjithë elementët e `set2`, që do të thotë se `set2` është një **nënset** i `set1`. Po ashtu, `set1` është një **mbiset** i `set2`. Në Python, mund të thërrasësh metodën `issubset` për të përcaktuar nëse një *set* është nënset i tjetrit. Formati i përgjithshëm:

```
set2.issubset(set1)
```

Metoda kthen `True` nëse `set2` është nënset i `set1`, përndryshe `False`. Mund të thërrasësh metodën `issuperset` për të përcaktuar nëse një *set* është mbiset i tjetrit. Formati i përgjithshëm:

```
set1.issuperset(set2)
```

Metoda kthen `True` nëse `set1` është mbiset i `set2`, përndryshe `False`. Demonstrim:

```
set1 = set([1, 2, 3, 4])
set2 = set([2, 3])
print(set2.issubset(set1)) # True
print(set1.issuperset(set2)) # True
```

Mund të përdorësh gjithashtu operatorin `<=` për të testuar nëse një *set* është nënset i një tjetri, dhe operatorin `>=` për të testuar nëse një *set* është mbiset i një tjetri. Formati i përgjithshëm me `<=` :

```
set2 <= set1
```

Shprehja kthen `True` nëse `set2` është nënset i `set1` , përndryshe `False` . Ndërsa me `>=` :

```
set1 >= set2
```

Shprehja kthen `True` nëse `set1` është mbiset i `set2` , përndryshe `False` . Demonstrim:

```
set1 = set([1, 2, 3, 4])
set2 = set([2, 3])

print(set2 <= set1) # True
print(set1 >= set2) # True
print(set1 <= set2) # False
```

1.2.11 Shprehje me *set* (*Set Comprehensions*)

Set comprehension është një shprehje që lexon një renditje elementësh hyrës dhe prodhon një *set*. Ato funksionojnë si *list comprehensions*, por *set*-et shkruhen me kllapa kaçurrelë `{}` (ndërsa listat me kllapa katrore `[]`).

Shembull — Kopjimi i një *set*-i

```
set1 = set([1, 2, 3, 4, 5])
set2 = {item for item in set1}
print(set1)
print(set2) # një kopje (rendi mund të ndryshojë)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
{1, 2, 3, 4, 5}
{1, 2, 3, 4, 5}
```

Shembull — Katrorët e elementëve

```
set1 = set([1, 2, 3, 4, 5])
set2 = {item**2 for item in set1}
print(set2) # {1, 4, 9, 16, 25} (rendi mund të ndryshojë)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
{1, 4, 9, 16, 25}
```

Shembull — Filtrim me `if`

Supozojmë se duam vetëm vlerat më të vogla se 10:

```
set1 = set([1, 20, 2, 40, 3, 50])
set2 = {item for item in set1 if item < 10}
print(set2) # {1, 2, 3} (rendi mund të ndryshojë)
```

Kur ekzekutohet, ky kod shfaq këtë output:

```
{1, 2, 3}
```

Shënime të dobishme

- **Set**-et ruajnë vetëm *vlera unike*, ndaj duplikimet hiqen automatikisht gjatë ndërtimit.
- Rendi i shfaqjes së elementëve në një *set nuk garantohet.*
- `iterable` mund të jetë listë, *tuple*, *string*, *set* etj.

1.3 Fjalorë të folezuar (*Nested Dictionaries*)

Përkufizim

Fjalor i folezuar quhet një fjalor që si *vlera* përmban një ose më shumë *fjalorë* të tjerë. Kjo lejon modelimin e të dhënave në nivele (*hierarki*), p.sh. *departament* → *punonjës* → *atribute.*

```
students = {
    'A101': {'name': 'Arta', 'scores': [90, 85, 92], 'active': True},
    'B202': {'name': 'Bledi', 'scores': [78, 92], 'active': False},
}
print(students['A101']['name']) # Arta
print(students['A101']['scores'][0]) # 90
```

1.3.1 Krijimi dhe përditësimi

Mund të shtosh një element të ri në nivelin e jashtëm dhe pastaj të plotësosh *fjalorin e brendshëm*:

```
students['C303'] = {}
students['C303']['name'] = 'Clea'
students['C303']['scores'] = [88, 91]
students['C303']['active'] = True
```

Këshillë praktike: `setdefault`

Kur do të shtosh në mënyrë të sigurt një nivel të brendshëm, përdor `setdefault`:

```
catalog = {}
catalog.setdefault('books', {}).setdefault('python', []).append('Fluent Python')
```

Kjo krijon automatikisht fjalorët/listat që mungojnë, pa ngritur `KeyError`.

1.3.2 Aksesim i sigurt me get

`dict.get` ndihmon të shmangësh `KeyError` kur një çelës mund të mungojë:

```
dept = {
    'IT': {'manager': 'Elira', 'headcount': 12},
    'HR': {'headcount': 4}
}

manager_it = dept.get('IT', {}).get('manager', 'N/A') # 'Elira'
manager_hr = dept.get('HR', {}).get('manager', 'N/A') # 'N/A'
manager_fin = dept.get('Finance', {}).get('manager', 'N/A') # 'N/A'
print(manager_it, manager_hr, manager_fin)
```

Kujdes me kopjimin (*shallow* vs *deep*)

`dict.copy()` bën *shallow copy*: fjalori i jashtëm kopjohet, por fjalorët/listat e brendshme *ndahen* (referenca të njëjta). Përdor `copy.deepcopy` për kopjim të thellë.

```
import copy
orig = {'A101': {'scores': [90, 95]}}
shallow = orig.copy()
deep = copy.deepcopy(orig)

shallow['A101']['scores'].append(100)
print(orig) # Ndryshohet edhe orig (shallow copy)
print(deep) # I pandryshuar
```

1.3.3 Iterimi në nivele

Mund të *çpaketosh* çifte çelës–vlerë dhe pastaj të hysh brenda:

```
for sid, info in students.items():
    name = info.get('name', 'Unknown')
    active = info.get('active', False)
    first = info.get('scores', [None])[0]
    print(sid, name, active, first)
```

1.3.4 Fshirja në nivele

Fshij një fushë të brendshme ose të gjithë hyrjen:

```
del students['B202']['active'] # heq vetëm fushën 'active'
if not students['B202']: # nëse bosh, hiqe gjithë hyrjen
    del students['B202']
```

1.3.5 *Nested* dictionary comprehensions

Mund të ndërtosh struktura të folezuara me *comprehensions*:

```
names = ['Ana', 'Besa']
levels = [1, 2, 3]
squares_by_name = {
    name: {n: n**2 for n in levels}
    for name in names
}
print(squares_by_name)
# {'Ana': {1: 1, 2: 4, 3: 9}, 'Besa': {1: 1, 2: 4, 3: 9}}
```

1.4 Serializimi i objekteve (*Pickling*)

Përkufizim

Serializim quajmë procesin e shndërrimit të një objekti në një rrjedhë bajtësh (*byte stream*) që mund të ruhet në një skedar dhe të rikthehet më vonë. Në Python, serializimi i objekteve quhet **pickling**, ndërsa rikthimi i tyre quhet **unpickling**. Moduli standard `pickle` ofron funksionet përkatëse.

1.4.1 Pse serializojmë?

- Ruajtje në disk të strukturave komplekse (`dict`, `set`, lista, `tuple`, etj.).
- Dërgim midis proceseve ose seancave të ndryshme të aplikacionit.
- *Cache* e përllogaritjeve të kushtueshme.

Siguria në radhë të parë

Mos bëj `pickle.load` me të dhëna nga burime të pabesueshme. `pickle` mund të ekzekutojë kod gjatë *unpickling*. Për shkëmbim të dhënash me palë të treta përdor formate të sigurta si `json`.

1.4.2 Hapat bazë për *pickling*

Hapat për të ruajtur një objekt në një skedar me `pickle` janë si më poshtë:

1) `import pickle`
2) Hap një skedar për **shkrim binar** me `'wb'`.
3) Thirr `pickle.dump(obj, file, protocol=pickle.HIGHEST_PROTOCOL)`.
4) Mbyll skedarin (ose përdor `with`).

```
import pickle

phonebook = {'Chris': '555-1111', 'Katie': '555-2222', 'Joanne': '555-3333'}

with open('phonebook.dat', 'wb') as f:
    pickle.dump(phonebook, f, protocol=pickle.HIGHEST_PROTOCOL)
```

1.4.3 Hapat bazë për *unpickling*

Hapat për të rikthyer një objekt nga një skedar me `pickle` janë si më poshtë:

1) `import pickle`
2) Hap skedarin për **lexim binar** me `'rb'`.
3) Thirr `obj = pickle.load(file)` për të rikthyer objektin.

```
import pickle

with open('phonebook.dat', 'rb') as f:
    pb = pickle.load(f)

print(pb)
# {'Chris': '555-1111', 'Joanne': '555-3333', 'Katie': '555-2222'}
```

Shënim

Mund të **dump**-osh disa objekte rresht në të njëjtin skedar. Gjatë leximit, thirr `pickle.load` në të njëjtën renditje derisa të kapësh `EOFError` (fund i skedarit).

Programi 1 - Pickling multiple objects (`pickle_objects.py`)

```
import pickle

def main():
    again = 'y'
```

```
    output_file = open('info.dat', 'wb') # hap për shkrim binar një herë
    try:
        while again.lower() == 'y':
            person = get_person()
            pickle.dump(person, output_file, protocol=pickle.HIGHEST_PROTOCOL)
            again = input('Enter more data? (y/n): ')
    finally:
        output_file.close()

def get_person():
    person = {}
    person['name'] = input('Name: ')
    person['age'] = int(input('Age: '))
    person['weight'] = float(input('Weight: '))
    return person
```

Programi 2 - Unpickling multiple objects (`unpickle_objects.py`)

```
import pickle

def main():
    end_of_file = False
    input_file = open('info.dat', 'rb')
    try:
        while not end_of_file:
            try:
                person = pickle.load(input_file)
                display(person)
            except EOFError:
                end_of_file = True
    finally:
        input_file.close()

def display(person):
    print('Name:', person['name'])
    print('Age:', person['age'])
    print('Weight:', person['weight'])
    print()
```

Këshilla praktike

- Përdor `with open(...)` për mbyllje automatike të skedarit.
- Përdor `protocol=pickle.HIGHEST_PROTOCOL` për skedarë më kompaktë dhe më të shpejtë.
- `pickle` është **specifik për Python** dhe jo i lexueshëm nga njerëzit. Për shkëmbim ndërgjuhësh përdor `json`.

1.5 Formati JSON dhe përdorimi në Python

Përkufizim

JSON (JavaScript Object Notation) është një *format tekstual, i lehtë dhe i ndërveprueshëm* për shkëmbim të dhënash. Është i pavarur nga gjuha, bazohet në Unicode/UTF-8 dhe përbëhet nga *objekte* (`{...}` me çifte *çelës–vlerë*) dhe *array* (`[...]` , renditje vlerash). Çelësat **duhet** të jenë *string* me thonjëza të dyfishta.

Pse JSON?

Ideale për: API, konfigurime, log-e, skedarë të lexueshëm nga njerëzit dhe të shumëgjuhshëm. Krahaso me `pickle` : JSON është më i sigurt për shkëmbim publik dhe ndërplatformë, ndërsa `pickle` ruan objekte Python (por s'është i portueshëm dhe ka rreziqe sigurie).

1.5.1 Hartimi i tipeve: JSON ↔ Python

JSON	Python (me `loads`)	JSON (me `dumps`)
`object` `{...}`	`dict`	`dict` → `object`
`array` `[...]`	`list`	`list` / `tuple` → `array`
`"text"`	`str`	`str` → `"text"`
`number`	`int` / `float`	`int` / `float` → `number`
`true` / `false`	`True` / `False`	`bool` → `true` / `false`
`null`	`None`	`None` → `null`

Rregulla të rëndësishme

- **Pa komente**, **pa presje fundore** (trailing commas).
- Çelësat **me thonjëza të dyfishta** `"key"` (jo `'key'`).
- Vlerat speciale si `NaN` , `Infinity` *nuk* janë pjesë e specifikimit standard.

1.5.2 Serializim/deserializim bazë

```python
import json

data = {
    "name": "Arta",
    "age": 20,
    "scores": [90, 85, 92],
    "active": True
}

# -> String JSON
s = json.dumps(data, ensure_ascii=False) # ruaj shkronjat shqipe pa \u-escape
print(s)

# <- Nga string JSON në objekte Python
obj = json.loads(s)
print(obj["name"], obj["scores"][0])
```

Për skedarë: `json.dump` / `json.load`

```python
# Writting in JSON file (pretty-printed with indentation)
with open("students.json", "w", encoding="utf-8") as f:
    json.dump(data, f, ensure_ascii=False, indent=2, sort_keys=True)

# Reading from JSON file
with open("students.json", "r", encoding="utf-8") as f:
    students = json.load(f)
```

Opsione të dobishme

`indent=2` për formatim të lexueshëm; `sort_keys=True` për rend të qëndrueshëm të çelësave; `ensure_ascii=False` për Unicode natyral (shqip).

1.5.3 Punë me data/kohë dhe tipe jo-standarde

JSON standard nuk njeh `datetime`. Serializojmë si `ISO 8601` dhe rikthejmë me `object_hook`.

```python
from datetime import datetime
import json

invoice = {"id": 101, "created_at": datetime(2025, 11, 5, 15, 0)}

def default(o):
    if isinstance(o, datetime):
        return o.isoformat() # p.sh. "2025-11-05T15:00:00"
    raise TypeError(f"Tipi nuk serializohet: {type(o)}")

s = json.dumps(invoice, default=default, ensure_ascii=False, indent=2)
print(s)
```

```python
from datetime import datetime
import json

def parse_dates(d):
    # Shembull i thjeshtë: çdo çelës që mbaron me _at lexohet si datetime
    for k, v in list(d.items()):
        if isinstance(v, str) and k.endswith("_at"):
            try:
                d[k] = datetime.fromisoformat(v)
            except ValueError:
                pass
    return d

obj = json.loads(s, object_hook=parse_dates)
print(type(obj["created_at"]), obj["created_at"])
```

1.5.4 Saktësia numerike me `decimal.Decimal`

```
import json, decimal

raw = '{"price": 19.99, "tax": 0.22}'
precise = json.loads(raw, parse_float=decimal.Decimal)
print(type(precise["price"]), precise["price"] * precise["tax"])
```

1.5.5 JSON me fjalorë të folezuar

Përshtatet natyrshëm me strukturat e seksionit "Fjalorë të folezuar":

```
import json
students = {
    "A101": {"name": "Arta", "scores": [90, 85, 92], "active": True},
    "B202": {"name": "Bledi", "scores": [78, 92], "active": False}
}

# -> JSON
s = json.dumps(students, ensure_ascii=False, indent=2)
print(s)

# <- JSON
back = json.loads(s)
print(back["A101"]["scores"][0]) # 90
```

1.5.6 Lexim i shpejtë i rrjedhave "një-JSON-për-rresht" (NDJSON)

```
import json

# skedar ku çdo rresht është një objekt JSON i veçantë
with open("events.ndjson", "r", encoding="utf-8") as f:
    for line in f:
        if line.strip():
            evt = json.loads(line)
            # përpuno ngjarjen 'evt'
```

1.5.7 Trajtimi i gabimeve

```
import json
bad = '{"a": 1,}' # presja fundore e paligjshme

try:
    json.loads(bad)
except json.JSONDecodeError as e:
    print("JSON i pavlefshëm:", e)
```

Kujdes me vlefshmërinë

- *Pa* komente `// ...` ose `/* ... */`.
- *Pa* presje fundore në objekte/array.
- Çelësat *duhet* të jenë `"string"` (jo numra, jo `None`).

Kur të përdorësh JSON vs `pickle`

JSON: kur do format të lexueshëm, të portueshëm dhe të sigurt për API/konfigurime/skedarë publikë.
`pickle` : cache lokale ose ruajtje/transferim objektesh specifike për Python (jo për input të pabesueshëm).

	pickle	**json**
Formati	binar, jo i lexueshëm	tekst, i lexueshëm
Portabiliteti	vetëm Python	shumëgjuhësh
Siguria	rrezik me burime të pabesueshme	më i sigurt për input publik
Shpejtësia/Tipet	i shpejtë, ruan objekte komplekse	i kufizuar te tipet bazë